GOOLLUCK CONSULTING LLP

SeatLock

Concurrent Train Seat Booking System

Role	Full Stack Developer Intern	**Expected effort**	5–7 hours
Submission window	Within 24 hours of receiving the assignment	**Demo**	Local run is mandatory

1. Objective

Build a train seat booking application where **only 20 seats are available, while 100 users may attempt to book at approximately the same time**. The system must manage seat availability, temporary seat locks, booking confirmation, and concurrent requests without allowing duplicate bookings.

Core success condition: even under simultaneous booking requests, no more than 20 unique seats can be confirmed, and no seat can be assigned to two users.

2. Mandatory Functional Requirements

A. Seat Selection and Availability

• Display one train coach containing 20 seats.

• Show each seat as Available, Locked, or Booked.

• Allow a user to select available seats and view current seat status fetched from the backend.

• Prevent selection of seats that are already booked or actively locked by another user.

B. Temporary Seat Locking

• When checkout begins, temporarily lock the selected seats for 2 minutes.

• Display a countdown timer for the active reservation.

• Allow the user to confirm or cancel the reservation.

• Release seats when a lock expires or is cancelled.

• Reject confirmation attempts for expired locks.

C. Booking Confirmation

• Provide a simulated payment/confirmation action; no real payment gateway is required.

• Mark seats as Booked and persist confirmed bookings in the database.

• Ensure the same seat cannot be booked twice.

D. Concurrent Booking — Core Challenge

• Handle multiple booking requests arriving at approximately the same time.

• Never confirm more than 20 seats in total.

• Return a clear response when a requested seat is unavailable.

• Prevent race conditions and maintain database consistency.

• Use appropriate transaction management, database locking, or atomic operations.

3. Technology Requirements

Layer	Required technology
Frontend	React.js
Backend	Java 17+ with Spring Boot
Database	PostgreSQL or MySQL
Persistence	Spring Data JPA / Hibernate
API	REST; use a basic Controller–Service–Repository structure

4. Mandatory Local Demonstration

The application must run locally on the candidate's machine. A public deployment is **not required**. During the demonstration, show the following:

• Select and temporarily lock an available seat.

• Confirm a booking and show the updated seat status.

• Cancel a reservation or allow the 2-minute lock to expire; verify that the seat becomes available again.

• Attempt to book a seat that is already locked or booked and show a clear error response.

• Simulate multiple simultaneous booking requests using a script, automated test, or API testing tool.

• Verify that no more than 20 seats can be confirmed and that no seat is assigned twice.

5. Submission Requirements

• GitHub repository containing the frontend and backend source code.

• README with local setup and execution instructions.

• Database schema or migration scripts.

• API documentation or a Postman collection.

• Screenshots or a short demonstration video.

• A brief explanation of how duplicate bookings and race conditions are prevented.

6. Evaluation Criteria

Criteria	Weight
Seat booking and temporary locking	**20%**
Concurrent request handling	**30%**
Database consistency and transactions	**20%**
React frontend and API integration	**15%**
Code quality and documentation	**10%**
Testing and error handling	**5%**
Total	**100%**

7. Bonus Features (Optional)

• Deploy the application to a public URL.

• Docker and Docker Compose setup.

• Automated tests for reservation expiry and concurrent requests.

• Booking history or a simple admin dashboard.

• Authentication and user accounts.

• Improved responsive UI/UX.

Bonus features are optional and will not compensate for incomplete core functionality. Prioritize correctness and data consistency over elaborate UI design.

8. Important Instructions

• Complete mandatory requirements before attempting optional features.

• The application must run locally and demonstrate the core functionality.

• You may use documentation and AI tools, but you should understand and be able to explain your implementation. Disclose significant AI-tool usage in the README.

• Do not commit passwords, API keys, or other sensitive credentials to the repository.

We look forward to reviewing your work.
Hiring Team · Goolluck Consulting LLP